# 8. Więzy integralności i operacje kaskadowe

System zarządzania czasem pracy — implementacja w PostgreSQL (schemat work_time). Opracowanie oparto na modelu 24 relacji z projektu oraz na aktualnym skrypcie DDL z kluczem Assignment.assignment_id. Wybrane akcje referencyjne przedstawione w tym rozdziale są rozszerzeniem wcześniejszego skryptu, a nie zapisami już istniejącymi w pierwotnym modelu.

## 8.1. Rodzaje więzów integralności

| Rodzaj | Realizacja SQL | Zastosowanie w projekcie |
|---|---|---|
| Integralność encji | PRIMARY KEY, NOT NULL | Unikalne identyfikatory 24 tabel; Assignment.assignment_id jest kluczem głównym. |
| Integralność referencyjna | FOREIGN KEY, REFERENCES | Powiązania Employee–Team, Contract–Employee, Assignment–Shift/Employee/Zone itd. |
| Integralność unikalności | UNIQUE, unikalny indeks | Login i employee_id w Account; (shift_id, employee_id) w Assignment; jedna opublikowana wersja Schedule dla tego samego okresu. |
| Integralność dziedzinowa | CHECK, NOT NULL, typy danych | Dodatnia pojemność Zone, etat (0;1], okresy, dozwolone statusy i typy zdarzeń. |

Więzy są egzekwowane przez silnik PostgreSQL podczas wstawiania i aktualizacji danych; więzy referencyjne kontrolują także skutki usuwania rekordów powiązanych. Przykładowo próba przypisania pracownika, którego employee_id nie istnieje, kończy się błędem klucza obcego.

```
CREATE TABLE "Assignment" (
  "assignment_id" INTEGER GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
  "shift_id" INTEGER NOT NULL REFERENCES "Shift" ("shift_id"),
  "employee_id" INTEGER NOT NULL REFERENCES "Employee" ("employee_id")
    ON DELETE RESTRICT,
  "zone_id" INTEGER NOT NULL REFERENCES "Zone" ("zone_id"),
  CONSTRAINT "uq_assignment_shift_employee"
    UNIQUE ("shift_id", "employee_id")
);
```

assignment_id ułatwia wskazanie konkretnego przydziału, natomiast UNIQUE(shift_id, employee_id) nadal gwarantuje, że w obrębie jednej zmiany pracownik wystąpi najwyżej raz, a więc będzie miał najwyżej jedną strefę pracy.

## 8.2. Dostępne działania referencyjne

PostgreSQL udostępnia pięć działań przy ON DELETE i ON UPDATE. Klauzula ON DELETE określa skutek usunięcia rekordu nadrzędnego; ON UPDATE — zmiany wartości jego klucza. Domyślna reguła dla obydwu zdarzeń to NO ACTION.

| Działanie | Znaczenie |
|---|---|
| NO ACTION | Operacja jest niedozwolona, jeśli przy sprawdzeniu klucza obcego pozostaje błędne odwołanie. W odróżnieniu od RESTRICT może podlegać odroczonej weryfikacji, jeżeli więź zdefiniowano jako odraczalną. |
| RESTRICT | Blokuje usunięcie/zmianę klucza nadrzędnego, gdy istnieją |

| Działanie | Znaczenie |
|---|---|
| | zależne rekordy; nie pozwala odroczyć tego sprawdzenia. |
| CASCADE | ON DELETE: usuwa zależne rekordy; ON UPDATE: przepisuje nową wartość klucza nadrzędnego do FK. |
| SET NULL | Ustawia odpowiednie kolumny FK rekordu zależnego na NULL; wymaga, by kolumna dopuszczała NULL. |
| SET DEFAULT | Ustawia wartość domyślną FK; musi ona spełniać ograniczenie FK albo być dopuszczalnym NULL. |

W projekcie zastosowano jawnie CASCADE, SET NULL i RESTRICT. W pozostałych relacjach pozostawiono domyślne NO ACTION. SET DEFAULT omówiono teoretycznie, lecz nie wdrożono: sztuczny „domyślny" pracownik, oddział czy zmiana nie odpowiadałby semantyce danych.

### 8.3. Działania kaskadowe zastosowane w projekcie

| Klucz obcy | ON DELETE | ON UPDATE | Uzasadnienie |
|---|---|---|---|
| AccountRole.account_id → Account | CASCADE | CASCADE | Usuwane są tylko przypisania ról do usuwanego konta. |
| AccountRole.role_id → Role | CASCADE | CASCADE | Usunięcie roli usuwa jej przypisania do kont. |
| RolePermission.role_id → Role | CASCADE | CASCADE | Usunięcie roli usuwa związki roli z uprawnieniami. |
| RolePermission.permission_id → Permission | CASCADE | CASCADE | Usunięcie uprawnienia usuwa przypisania do ról. |
| BranchDayOff.branch_id → Branch | CASCADE | CASCADE | Usunięcie oddziału usuwa tylko wiązania z dniami wolnymi (jeśli inne FK pozwolą). |
| BranchDayOff.day_off_id → DayOff | CASCADE | CASCADE | Usunięcie dnia wolnego usuwa tylko powiązania z oddziałami. |
| Employee.manager_id → Employee | SET NULL | CASCADE | Zerowanie opcjonalnego odwołania do usuniętego przełożonego. |
| Contract.employee_id → Employee | RESTRICT | NO ACTION | Blokada usunięcia pracownika z zachowaną umową. |
| Assignment.employee_id → Employee | RESTRICT | NO ACTION | Blokada usunięcia pracownika z przydziałem do zmiany. |
| Registration.employee_id → Employee | RESTRICT | NO ACTION | Blokada usunięcia pracownika z rejestracjami czasu pracy. |

W tabelach pośrednich kaskada usuwa jedynie skojarzenia, nie obiekty docelowe. Przykładowo usunięcie roli usuwa wiersze AccountRole i RolePermission, lecz nie usuwa pracowników, kont ani samych uprawnień. Z kolei dla Contract, Assignment i Registration chronimy dane historyczne, zamiast usuwać je automatycznie.

```
ALTER TABLE "RolePermission"
  ADD CONSTRAINT "fk_rolepermission_role"
  FOREIGN KEY ("role_id") REFERENCES "Role" ("role_id")
  ON DELETE CASCADE ON UPDATE CASCADE;
```

```
ALTER TABLE "Employee"
  ADD CONSTRAINT "fk_employee_manager"
  FOREIGN KEY ("manager_id") REFERENCES "Employee" ("employee_id")
  ON DELETE SET NULL ON UPDATE CASCADE;
```

Są to fragmenty ilustrujące nową definicję więzów; podczas migracji wcześniejsze klucze obce trzeba najpierw usunąć, a następnie dodać ponownie. Gotowy, pełny zestaw poleceń znajduje się w pliku 03_migracja_wiezow_integralnosci.sql.

Uwaga praktyczna: zwykłe usuwanie pracowników nie jest zalecane. Projekt przewiduje oznaczanie ich jako archiwalnych (Employee.archived). SET NULL dla manager_id może zadziałać tylko wtedy, gdy inne klucze obce nie zablokują usunięcia przełożonego.

### 8.4. Zastosowanie instrukcji CHECK

CHECK definiuje warunek logiczny obliczany dla pojedynczego rekordu. W dotychczasowym skrypcie znajduje się 16 nazwanych ograniczeń CHECK. Jeśli INSERT/UPDATE narusza warunek (wyrażenie zwraca FALSE), operacja jest odrzucana.

| Tabela | Nazwa ograniczenia | Sprawdzany warunek i cel |
|---|---|---|
| Zone | ck_zone_capacity | capacity > 0 — Pojemność strefy musi być dodatnia. |
| Employee | ck_employee_not_own_manager | manager_id IS NULL OR manager_id <> employee_id — Zakaz wskazania siebie jako bezpośredniego przełożonego. |
| Contract | ck_contract_dates | end_date IS NULL OR end_date >= start_date — Umowa bezterminowa lub nieujemny okres obowiązywania. |
| Contract | ck_contract_fraction | employment_fraction > 0 AND <= 1 — Wymiar etatu w zakresie (0; 1]. |
| Annex | ck_annex_number | annex_number > 0 — Dodatni numer aneksu. |
| Annex | ck_annex_fraction | employment_fraction > 0 AND <= 1 — Wymiar etatu po zmianie w zakresie (0; 1]. |
| Annex | ck_annex_dates | end_date IS NULL OR end_date >= effective_from — Koniec obowiązywania aneksu nie może poprzedzać jego wejścia w życie. |
| Schedule | ck_schedule_dates | period_to >= period_from — Poprawna kolejność dat grafiku. |
| Schedule | ck_schedule_version | version_number > 0 — Dodatni numer wersji grafiku. |
| Schedule | ck_schedule_status | status IN ('draft', 'published', 'archived') — Tylko dozwolone statusy grafiku. |
| Shift | ck_shift_times | end_time > start_time — Zmiana musi kończyć się później, niż się zaczęła. |
| Availability | ck_availability_times | available_to > available_from — Prawidłowy, niepusty przedział dostępności. |
| Absence | ck_absence_dates | end_date >= start_date — Koniec nieobecności nie wcześniej niż początek. |
| Absence | ck_absence_status | status IN ('registered', 'cancelled') — Tylko zarejestrowana lub anulowana nieobecność. |
| Registration | ck_registration_event_type | event_type IN (work_start, work_end, break_start, break_end) — Dozwolone |

| Tabela | Nazwa ograniczenia | Sprawdzany warunek i cel |
|---|---|---|
| | | typy zdarzenia rejestracji pracy. |
| Registration | ck_registration_verification | verification_status IN (unverified, valid, needs_explanation) — Dozwolone wyniki weryfikacji rejestracji. |

```
CONSTRAINT "ck_zone_capacity" CHECK ("capacity" > 0),
CONSTRAINT "ck_contract_fraction" CHECK (
   "employment_fraction" > 0 AND "employment_fraction" <= 1
),
CONSTRAINT "ck_shift_times" CHECK ("end_time" > "start_time"),
CONSTRAINT "ck_schedule_status" CHECK (
   "status" IN ('draft', 'published', 'archived')
)
```

Przykład: INSERT do Zone z capacity = 0 zostanie odrzucony. Próba zapisania zmiany, której end_time nie jest późniejsze od start_time, również zakończy się błędem. Dzięki nazwom ck_* PostgreSQL wskazuje konkretny naruszony warunek.

Ważne: CHECK uznaje wyrażenie za spełnione także wtedy, gdy ma ono wynik UNKNOWN (NULL). Dlatego w kolumnach obowiązkowych — np. capacity, employment_fraction, start_time — zastosowano dodatkowo NOT NULL. Dla zakończenia umowy end_date dopuszczono NULL świadomie: oznacza brak wskazanej daty końcowej.

### 8.5. Ograniczenia CHECK a złożone reguły biznesowe

CHECK nie zastępuje walidacji wymagającej analizy innych wierszy lub tabel. W PostgreSQL typowy CHECK powinien odwoływać się tylko do wartości jednego aktualnie zapisywanego wiersza. Pozostałe reguły wymagają np. indeksów UNIQUE/EXCLUDE, funkcji z triggerami albo logiki aplikacyjnej.

| Wymaganie projektu | Stan w SQL DDL / wymagane uzupełnienie |
|---|---|
| F12 — brak cykli przełożonych | CHECK blokuje jedynie pracownika jako własnego przełożonego; cykl A → B → A wymaga odrębnego sprawdzenia. |
| F20 — zgodność przydziałów | UNIQUE blokuje podwójny przydział do tej samej zmiany, ale nie sprawdza nakładania się różnych zmian, absencji, dostępności, ważności umowy i pojemności stref. |
| F21–F22 — publikacja i wersje | Częściowy indeks unikalny nie dopuszcza dwóch opublikowanych wersji dla identycznego działu i identycznych granic okresu; nie eliminuje okresów częściowo zachodzących. |
| F24 — deklaracje dostępności | CHECK pilnuje kolejności dat; brak zakazu nakładających się/sprzecznych deklaracji tej samej osoby. |
| F30–F32 — sekwencja rejestracji | CHECK ogranicza typ i status zdarzenia; nie weryfikuje par start/koniec ani czasu pracy po odjęciu przerw. |
| F36 — audyt zmian | Tabela Audit przechowuje strukturę danych audytu; automatyczne wyzwalacze zapisujące operacje nie są częścią tego etapu. |

### 8.6. Implementacja i sprawdzenie więzów

Pliki SQL opracowano dla PostgreSQL 14+ i schematu work_time. Udostępniono dwie alternatywne ścieżki wdrożenia. Nie należy wykonywać pełnego CREATE TABLE ponownie na istniejącej bazie.

| Stan bazy | Sposób postępowania |
|---|---|
| Nowa, pusta baza | Uruchomić baza_postgresql_z_integralnoscia.sql, a następnie 04_testy_wiezow_integralnosci.sql. |
| Baza utworzona wcześniej | Uruchomić tylko 03_migracja_wiezow_integralnosci.sql, a następnie 04_testy_wiezow_integralnosci.sql. |

```
-- Wariant A: nowa baza
psql -X -U postgres -d ewidencja_czasu_pracy -v ON_ERROR_STOP=1 \
  -f baza_postgresql_z_integralnoscia.sql

-- Wariant B: istniejąca baza (ZAMIAST wariantu A)
psql -X -U postgres -d ewidencja_czasu_pracy -v ON_ERROR_STOP=1 \
  -f 03_migracja_wiezow_integralnosci.sql

-- Weryfikacja po A albo B
psql -X -U postgres -d ewidencja_czasu_pracy -v ON_ERROR_STOP=1 \
  -f 04_testy_wiezow_integralnosci.sql
```

Skrypt testów wypisuje katalog obowiązujących kluczy obcych wraz z akcjami ON DELETE/ON UPDATE i liczbę ograniczeń CHECK. Następnie sprawdza przykładowe naruszenia CHECK, UNIQUE i FOREIGN KEY, działanie CASCADE (usunięcie i aktualizacja), SET NULL i RESTRICT. Dane wstawiane w testach są wycofywane instrukcją ROLLBACK. Uwaga: wartości sekwencji IDENTITY mogą się zwiększyć także po wycofaniu transakcji.

Zakładany rezultat: 24 tabele, 16 nazwanych CHECK i 10 jawnie opisanych w rozdziale kluczy obcych z wybranymi akcjami. Pozostałe klucze obce działają z domyślnym NO ACTION. Nie przeprowadzono uruchomienia na rzeczywistej instancji PostgreSQL w środowisku przygotowania plików; dlatego komunikaty NOTICE OK są oczekiwanym wynikiem testów, a nie raportem z wykonanego wdrożenia.

## 8.7. Wnioski

Opracowane więzy integralności zabezpieczają podstawową spójność struktury danych i najważniejsze reguły dotyczące pojedynczych rekordów. Wybrane kaskady usprawniają zarządzanie relacjami wiele-do-wielu, bez usuwania historycznych danych o zatrudnieniu i czasie pracy. Wprowadzenie klucza głównego assignment_id nie osłabia unikalności przydziału, ponieważ zachowano ograniczenie UNIQUE(shift_id, employee_id). Pełne reguły kolizji harmonogramów, hierarchii służbowej, pojemności stref i audytu wymagają oddzielnej implementacji.

*Źródła: „Bazy projekt-2.pdf”, rozdz. 1–5; „bazy projekt (1).sql” i poprzednia wersja baza_postgresql.sql; PostgreSQL Documentation (Constraints): https://www.postgresql.org/docs/14/ddl-constraints.html.*